

D4: Analysis and Topology

Structures, filters, limits, derivatives, integrals, and proof workflow

Summer School on Lean

July 16, Thursday

Learning Goals

- read Mathlib analysis statements through their required structures;
- compare undergraduate epsilon language with Mathlib's filter language;
- read sequence, function, punctured, one-sided, and infinite limits as `Tendsto`;
- connect `ContinuousAt` and `Continuous` with preservation of limits;
- distinguish `deriv f x` from `HasDerivAt f c x`;
- recognize interval integrals and use a practical Mathlib proof workflow.

Four-Part Outline

Part I	Structures real numbers, type class assumptions, metric spaces, topology, neighborhoods
Part II	Limits epsilon-N, epsilon-delta, filters, Tendsto, continuity
Part III	Derivatives deriv, HasDerivAt, derivative rules and examples
Part IV	Integrals interval integrals, FTC map, Mathlib search and proof repair

Undergraduate Route and Mathlib Route

Undergraduate analysis

start with $\mathbb{R}$, absolute values, and inequalities

define limits by epsilon-N and epsilon-delta formulas

topology often appears later as an abstraction

Mathlib analysis

start with a type carrying the needed structures

express limits uniformly with `Tendsto`

topology and filters are infrastructure for analysis

Epsilon language is not discarded; it is recovered from `MetricSpace` statements.

Mathlib analysis is built from reusable structures.

- algebraic structure: operations and laws;
- order structure: inequalities, intervals, monotonicity;
- topological structure: neighborhoods, continuity, limits;
- metric structure: distance and epsilon-style statements;
- filters: a uniform language for convergence.

The point is not to memorize the hierarchy, but to read what a theorem needs.

The Real Line Carries Many Structures

```
import Mathlib

#check Real
#check abs
#check Set.Icc

#synth Field Real
#synth LinearOrder Real
#synth TopologicalSpace Real
#synth MetricSpace Real
```

- The real line is a familiar object with many registered instances.
- Type class search supplies algebraic, order, topological, and metric structure.
- Many analysis theorems are stated for abstract spaces, not only for `Real`.

Live Lean: `codeD4/D4_AnalyticStructures.lean`

Reading Structure Assumptions

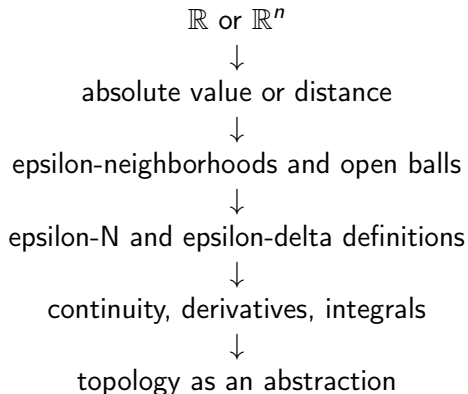
A theorem may look abstract because it advertises the structures it needs.

Assumption	Mathematical meaning
<code>[TopologicalSpace X]</code>	open sets, neighborhoods, topological limits
<code>[MetricSpace X]</code>	distance, open balls, epsilon-style statements
<code>[LinearOrderedField X]</code>	ordered field algebra and inequalities
<code>[NormedAddCommGroup X]</code>	norm, distance from subtraction, normed-space analysis

For a concrete type such as `Real`, Lean usually finds these instances automatically.

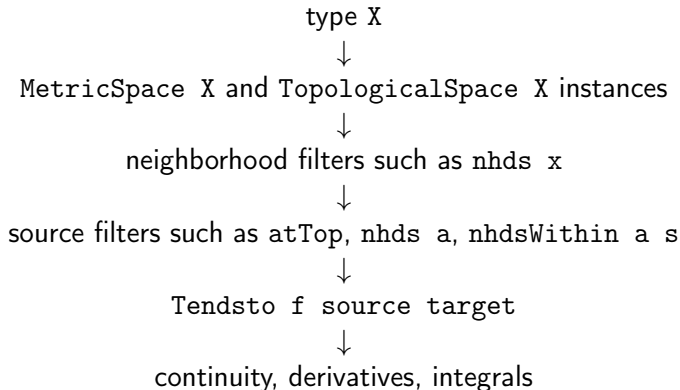
Undergraduate Convergence Route

A first analysis course often follows this route:



Mathlib Convergence Route

Mathlib usually organizes the same ideas in the opposite direction:



Vocabulary Translation

Undergraduate language	Mathlib language	Reading
$ x - y $ or distance	<code>dist x y</code>	distance between two points
epsilon open ball	<code>Metric.ball x eps</code>	points within radius eps
x near a	<code>nhds a</code>	neighborhood filter of a
$n \rightarrow \infty$	<code>atTop</code>	eventually for large indices
$x \rightarrow a$ within a set	<code>nhdsWithin a s</code>	restricted approach
$u_n \rightarrow L$	<code>Tendsto u ...</code>	sequence convergence
$\lim_{x \rightarrow a} f(x) = L$	<code>Tendsto f ...</code>	function limit

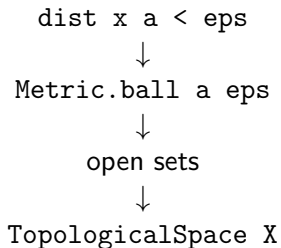
Metric Spaces: Distance and Balls

```
#check dist  
#check Metric.ball  
#check Metric.mem_ball
```

- A `MetricSpace` X supplies a distance function.
- In `Real`, `dist x a` corresponds to $|x - a|$.
- Epsilon-delta language is naturally metric-space language.

From Balls to Open Sets

In a metric space, open balls generate the usual topology.



Mathlib often states results at the topological level, then specializes them to metric spaces.

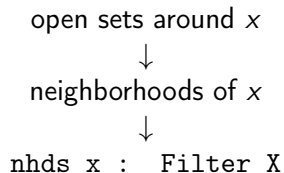
Topological Spaces

Object	Role in analysis
open sets	topological description of local information
neighborhoods	sets containing enough points near x
$\text{nhds } x$	filter of neighborhoods of x
Continuous f	preimages of opens are open, equivalently limits are preserved

Topology appears early in Mathlib because it supports many spaces beyond metric spaces.

Neighborhood Filters

The neighborhood filter $\text{nhds } x$ packages all neighborhoods of x .



Informally, a property holds along $\text{nhds } x$ if it holds sufficiently close to x .

What Is a Filter?

A filter is a way to say which sets are “large enough” or “close enough.”

- For `atTop` on $\mathbb{N}$, large sets contain all sufficiently large numbers.
- For `nhds a`, large sets contain a neighborhood of a .
- For `nhdsWithin a s`, large sets contain points near a , but only inside s .

A property is eventually true along a filter if it holds on one of these large sets.

A collection of large sets should satisfy:

- ① the whole space is large;
- ② if A is large and $A \subseteq B$, then B is large;
- ③ if A and B are large, then $A \cap B$ is large.

These are exactly the rules we expect from phrases like:

sufficiently large, sufficiently close, eventually true.

Filter: Formal Definition

A filter F on a type α is a collection

$$F.\text{sets} \subseteq \mathcal{P}(\alpha)$$

satisfying:

$$\alpha \in F.\text{sets},$$

$$x \in F.\text{sets} \wedge x \subseteq y \implies y \in F.\text{sets},$$

$$x \in F.\text{sets} \wedge y \in F.\text{sets} \implies x \cap y \in F.\text{sets}.$$

- `sets` is the collection of large sets.
- Mathlib allows a degenerate filter; nontriviality is expressed separately by `NeBot`.

Neighborhood Filter in Mathlib

The actual Mathlib definition of `nhds` is written using an infimum of principal filters, but the main reading lemma is:

```
#check nhds
#check mem_nhds_iff
```

$$s \in \mathcal{N}(x) \iff \exists t, t \subseteq s \wedge \text{IsOpen}(t) \wedge x \in t.$$

- A neighborhood of x is any set containing an open set around x .
- This is the concrete topological meaning behind `nhds x`.

Eventually

eventually x in F , $P\ x$

means:

$$\{x \mid P(x)\} \in F$$

- Along atTop : Px holds for all sufficiently large x .
- Along $\text{nhds } a$: Px holds for all x sufficiently close to a .

Two Basic Eventually Lemmas

```
#check eventually_atTop  
#check Metric.eventually_nhds_iff_ball
```

$$(\forall^f n \text{ in } \text{atTop}, P(n)) \iff \exists N, \forall n \geq N, P(n).$$

$$(\forall^f y \text{ in } \mathcal{N}(x), P(y)) \iff \exists \varepsilon > 0, \forall y \in B(x, \varepsilon), P(y).$$

Common Filters in Analysis

```
#check Filter
#check atTop
#check nhds
#check nhdsWithin
```

Filter	Natural-language reading
atTop	sufficiently large inputs
nhds a	sufficiently close to a
nhdsWithin a s	sufficiently close to a , while staying in s

The Route to Tendsto

MetricSpace X gives distance and balls
↓
TopologicalSpace X gives neighborhoods
↓
nhds x : Filter X describes closeness to x
↓
Tendsto f source target describes convergence

The goal of this part is to compare two languages:

- epsilon language: concrete and familiar;
- filter language: abstract and uniform;
- Mathlib usually proves limits by composing Tendsto lemmas.

Why Filters?

Situation	Traditional notation	Filter statement
sequence limit	$u_n \rightarrow L$	<code>Tendsto u atTop (nhds L)</code>
function limit	$x \rightarrow a$	<code>Tendsto f (nhds a) ...</code>
punctured limit	$x \rightarrow a, x \neq a$	<code>Tendsto f (nhdsWithin a s) ...</code>
right limit	$x \rightarrow a+$	<code>source: nhdsWithin a (Set.Ioi a)</code>
limit at infinity	$x \rightarrow +\infty$	<code>Tendsto f atTop (nhds L)</code>

The Shape of Tendsto

```
Tendsto f l1 l2
```

Piece	Meaning
f	the function being studied
l1	how inputs approach
l2	how outputs should approach

Do not read Tendsto as only a function limit; it is a general convergence statement.

Tendsto: Formal Definition

In Mathlib:

```
def Tendsto (f : alpha -> beta)
  (l1 : Filter alpha) (l2 : Filter beta) :=
  l1.map f <= l2
```

Natural-language reading:

every target-large set has source-large preimage.

Informally:

$$V \in l_2 \implies f^{-1}(V) \in l_1.$$

Source Filter and Target Filter

input side	function	output side
11	$\xrightarrow{f}$	12
atTop	$\xrightarrow{u}$	nhds L
nhds a	$\xrightarrow{f}$	nhds L

A common beginner mistake is to put the output neighborhood on the source side.

Undergraduate statement:

$$u_n \rightarrow L \iff \forall \varepsilon > 0, \exists N, \forall n \geq N, |u_n - L| < \varepsilon.$$

Metric-space version:

$$\forall \varepsilon > 0, \exists N, \forall n \geq N, \text{dist}(u_n, L) < \varepsilon.$$

Mathlib statement:

`Tendsto u atTop (nhds L)`

Reading:

- `atTop`: the index n is eventually large;
- `nhds L`: the output is eventually inside every neighborhood of L ;
- together: u_n tends to L .

Function Limits: Epsilon-Delta

Undergraduate statement:

$$\lim_{x \rightarrow a} f(x) = L$$

means:

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon.$$

Function Limits: Metric Form

In a metric space:

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, \text{dist}(x, a) < \delta \Rightarrow \text{dist}(f(x), L) < \varepsilon.$$

This is the metric expansion of a topological/filter statement.

Mathlib statement:

$$\text{Tendsto } f \text{ (nhds } a) \text{ (nhds } L)$$

Reading:

- input side: x approaches a ;
- output side: $f(x)$ approaches L ;
- Tendsto packages the epsilon-delta quantifiers.

Punctured Limits

Undergraduate punctured condition:

$$0 < |x - a| \quad \text{and} \quad |x - a| < \delta.$$

Mathlib reading:

`Tendsto f (nhdsWithin a s) (nhds L)`

For a punctured limit, s is a set excluding the point a .

One-Sided and Infinite Limits

Situation	Source filter
$x \rightarrow a$ from the right	<code>nhdsWithin a (Set.Ioi a)</code>
$x \rightarrow a$ from the left	<code>nhdsWithin a (Set.Iio a)</code>
$x \rightarrow +\infty$	<code>atTop</code>
$x \rightarrow -\infty$	<code>atBot</code>
sequence $n \rightarrow \infty$	<code>atTop on $\mathbb{N}$</code>

The template `Tendsto f source target` stays the same.

Metric Equivalence: Target Neighborhood

```
#check Metric.tendsto_nhds
```

$$\text{Tendsto } u \text{ l (nhds } a) \iff \forall \varepsilon > 0, \forall^f x \text{ in } I, \text{dist}(u(x), a) < \varepsilon.$$

With $I = \text{atTop}$, this becomes the usual epsilon-N language.

Metric Equivalence: Function Limits

```
#check Metric.tendsto_nhds_nhds
```

`Tendsto f (nhds a) (nhds b)`

is equivalent to:

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, \text{dist}(x, a) < \delta \Rightarrow \text{dist}(f(x), b) < \varepsilon.$$

Metric Equivalence: Restricted Limits

```
#check Metric.tendsto_nhdsWithin_nhds  
#check Metric.continuousAt_iff
```

- `Metric.tendsto_nhdsWithin_nhds` gives the epsilon-delta form with $x \in s$.
- Punctured limits are modeled by choosing s to exclude the point.
- `Metric.continuousAt_iff` gives the epsilon-delta definition of continuity at a point.

Filter language is the general interface; epsilon language is the metric expansion.

In Lean: Inspect the Concepts

```
open Filter
open scoped Topology

#check Filter.Tendsto
#check atTop
#check nhds
#check nhdsWithin
#check tendsto_const_nhds
#check tendsto_id
```

- Use `#check` to learn the type of a concept or theorem.
- This is often the fastest way to read Mathlib analysis code.

Live Lean: `codeD4/D4_LimitsContinuity.lean`

In Lean: Compose Tendsto Lemmas

```
#check tendsto_const_nhds
#check tendsto_id
#check Tendsto.add
#check Tendsto.mul
#check Continuous.tendsto
```

- We usually do not unfold epsilon-delta definitions.
- We combine reusable theorems about `Tendsto`.
- Algebraic operations on limits are API lemmas.

Continuity at a Point

Undergraduate reading:

$$x \rightarrow a \implies f(x) \rightarrow f(a).$$

Mathlib concepts:

```
ContinuousAt f a  
Tendsto f (nhds a) (nhds (f a))
```


Continuity Everywhere

- `ContinuousAt f a`: continuity at one point.
- `Continuous f`: continuity at every point.
- Continuous functions preserve limits.
- Many proofs use continuity lemmas instead of proving limits directly.

Common examples:

- constant functions;
- identity function;
- sums and products of continuous functions;
- compositions of continuous functions.

Common Reading Mistakes

- Reversing source and target filters in `Tendsto`.
- Treating `nhds a` as a single set rather than a filter of neighborhoods.
- Forgetting that `nhdsWithin a s` restricts the approach to a set.
- Expecting every Mathlib proof to unfold epsilon-delta definitions.
- Ignoring typeclass assumptions such as `MetricSpace X`.

20-minute break

The goal is to read and use Mathlib's derivative API.

- undergraduate language: $f'(x) = c$;
- proof-friendly Mathlib language: `HasDerivAt f c x`;
- value-level Mathlib language: `deriv f x`;
- derivative calculations are assembled from reusable rules.

Undergraduate vs. Mathlib

Undergraduate notation	Mathlib expression
$f'(x) = c$	<code>HasDerivAt f c x</code>
derivative value	<code>deriv f x</code>
derivative function	<code>fun x => deriv f x</code>
derivative rule	theorem about <code>HasDerivAt</code>

The proposition `HasDerivAt` is often the better interface for proofs.

Two Interfaces

```
#check deriv  
#check HasDerivAt
```

- `deriv f x` is a value.
- `HasDerivAt f c x` is a proposition.
- Rules such as `sum`, `product`, and `chain rule` usually produce `HasDerivAt` statements.

Live Lean: `codeD4/D4_Differentiation.lean`

What Does HasDerivAt Say?

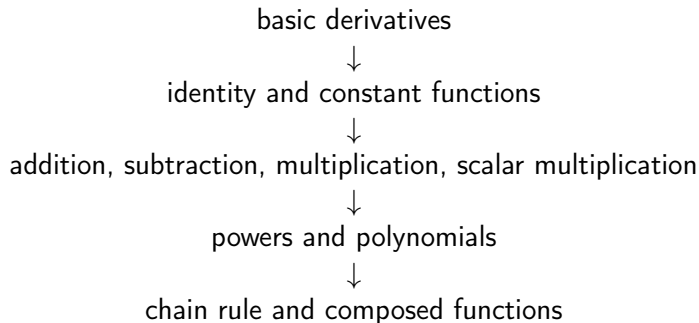
Informally:

`HasDerivAt f c x` means $f(x + h) = f(x) + ch + o(h)$.

For this lecture:

- read it as “the derivative of f at x is c ”;
- use API rules instead of unfolding the definition;
- keep the focus on theorem statements and proof composition.

Derivative Rule Map



Basic Derivative API

```
#check hasDerivAt_id  
#check hasDerivAt_const  
#check HasDerivAt.add  
#check HasDerivAt.mul  
#check HasDerivAt.comp
```

- Start from derivative facts for simple functions.
- Combine them with methods such as `.add`, `.mul`, and `.comp`.
- Use `simp` to align the final expression with the goal.

Identity and Constant Functions

```
example (x : Real) :  
  HasDerivAt (fun y : Real => y) 1 x := by  
  simpa using (hasDerivAt_id x)  
  
example (c x : Real) :  
  HasDerivAt (fun _ : Real => c) 0 x := by  
  simpa using (hasDerivAt_const (x := x) (c := c))
```

These are the derivative API's base cases.

```
#check hasDerivAt_pow  
#check deriv_pow
```

Mathlib knows the usual rule:

$$\frac{d}{dx}x^n = nx^{n-1}.$$

In classroom examples, powers give good low-friction demonstrations.

Example: $x \mapsto x^2$

```
example (x : Real) :  
  HasDerivAt (fun y : Real => y ^ 2) (2 * x) x := by  
  simpa using (hasDerivAt_pow 2 x)
```

- The mathematical derivative is $2x$.
- The Lean proof delegates the calculation to `hasDerivAt_pow`.
- `simpa` performs the small normalization needed for the exact goal.

Example: $x \mapsto x^3 + x$

```
example (x : Real) :  
  HasDerivAt (fun y : Real => y ^ 3 + y)  
    (3 * x ^ 2 + 1) x := by  
  have hpow :  
    HasDerivAt (fun y : Real => y ^ 3) (3 * x ^ 2) x := by  
    simpa using (hasDerivAt_pow 3 x)  
  have hid : HasDerivAt (fun y : Real => y) 1 x := by  
    simpa using (hasDerivAt_id x)  
  simpa using hpow.add hid
```

From HasDerivAt to deriv

```
example (x : Real) :  
  deriv (fun y : Real => y ^ 2) x = 2 * x := by  
  simp  
  
example (x : Real) :  
  deriv (fun y : Real => y ^ 3 + y) x = 3 * x ^ 2 + 1 := by  
  simp
```

- `deriv` is convenient for final value statements.
- For proof construction, `HasDerivAt` often gives more control.

Chain Rule Map

For $x \mapsto (x + 1)^2$:

$$g(x) = x + 1, \text{ derivative } 1$$



$$h(z) = z^2, \text{ derivative } 2z$$



$$h(g(x)) = (x + 1)^2$$



$$\text{chain rule: derivative } 2(x + 1)$$

This is a good live search exercise rather than a memorization target.

Chain Rule Search Targets

```
#check HasDerivAt.comp  
#check HasDerivAt.add  
#check hasDerivAt_pow  
#check hasDerivAt_id  
#check hasDerivAt_const
```

- Ask: which theorem has the conclusion shape we need?
- If algebraic expressions differ syntactically, try `simp` or a normalization tactic.
- This example models realistic Mathlib navigation.

This part gives a map of the integration API.

- interval integral notation;
- simple examples such as constant functions;
- the fundamental theorem of calculus as an API bridge;
- proof workflow for finding and repairing Mathlib proofs.

Why Integration Needs More Infrastructure

Differentiation can often be taught through pointwise rules.

Integration in Mathlib is built to cover:

- real and vector-valued functions;
- oriented intervals;
- measure-theoretic integration;
- integrability assumptions;
- fundamental theorem of calculus statements.

The lecture goal is to learn the map, not the whole measure theory.

Interval Integral Notation

```
#check intervalIntegral  
#check IntervalIntegrable
```

Mathematical notation:

$$\int_a^b f(x) dx$$

Mathlib notation:

```
integral x in a..b, f x
```

Live Lean: `codeD4/D4_Integration.lean`

Oriented Intervals

The interval integral is oriented:

$$\int_a^b f(x) dx = - \int_b^a f(x) dx.$$

Important special case:

$$\int_a^a f(x) dx = 0.$$

This is why theorem names include words such as same and symm.

```
#check intervalIntegral.integral_same  
#check intervalIntegral.integral_symm  
#check intervalIntegral.integral_const  
#check intervalIntegral.integral_add
```

- Use `#check` to inspect exact assumptions.
- Some linearity theorems include integrability hypotheses.
- Constant-function examples are good classroom entry points.

Example: Constant Function

```
example (a b c : Real) :  
  (integral x in a..b, c) = (b - a) * c := by  
  simpa using  
    (intervalIntegral.integral_const (a := a) (b := b) (c := c))
```

This is a good first integral theorem because the statement matches the undergraduate formula.

Example: Same Endpoints

```
example (a : Real) (f : Real -> Real) :  
  (integral x in a..a, f x) = 0 := by  
  simp
```

The simplifier knows many basic interval-integral facts.

Informal rules:

$$\int_a^b (f + g) = \int_a^b f + \int_a^b g, \quad \int_a^b cf = c \int_a^b f.$$

Mathlib versions may require:

- `IntervalIntegrable f volume a b`;
- `IntervalIntegrable g volume a b`;
- scalar and normed-space structure.


```
#check intervalIntegral.integral_add  
#check intervalIntegral.integral_sub  
#check intervalIntegral.integral_const_mul  
#check intervalIntegral.integral_mul_const
```

- The theorem statement tells you the hypotheses.
- Do not guess the assumptions; inspect them.
- This is typical for analysis APIs in Mathlib.

FTC I: Integral as an Antiderivative

Undergraduate shape:

$$F(x) = \int_a^x f(t) dt \implies F'(x) = f(x).$$

Mathlib connects:

- continuity of f ;
- the interval integral $x \mapsto \int_a^x f(t) dt$;
- a derivative statement such as `HasDerivAt`.

FTC I: API Entry Points

```
#check intervalIntegral.integral_hasDerivAt_right  
#check intervalIntegral.deriv_integral_right  
#check intervalIntegral.integral_hasDerivAt_left
```

- These are theorem names to inspect, not names to memorize.
- The exact theorem depends on endpoint direction and differentiability form.

FTC II: Integral of a Derivative

Undergraduate shape:

$$F' = f \implies \int_a^b f(x) dx = F(b) - F(a).$$

In Lean, the statement must say precisely:

- where F has derivative f ;
- which interval is used;
- which continuity or integrability hypotheses are available.

FTC II: API Entry Points

```
#check intervalIntegral.integral_eq_sub_of_hasDerivAt
#check intervalIntegral.integral_deriv_eq_sub
#check intervalIntegral.integral_deriv_eq_sub'
```

- Use #check before trying to apply an FTC theorem.
- The theorem name often reveals the conclusion shape.

When reading an FTC theorem, ask:

- ① Is the conclusion about `HasDerivAt`, `deriv`, or an integral equality?
- ② Is the integral from a to x , or from x to a ?
- ③ Does the theorem require continuity, differentiability, or interval integrability?
- ④ Are endpoint or interval assumptions hidden in the statement?

A practical Mathlib workflow:

- ① read the goal and local context;
- ② inspect definitions and theorem statements with `#check`;
- ③ search for API lemmas;
- ④ start with a small version of the target;
- ⑤ use Lean errors to repair theorem names and assumptions;
- ⑥ keep the final proof small and verified.

Live Lean: `codeD4/D4_ProofWorkflow.lean`

Workflow Example: Continuity to Limit

```
#check Continuous.tendsto
#check continuous_id
#check continuous_const

example (a c : Real) :
  Tendsto (fun x : Real => x + c) (nhds a) (nhds (a + c)) := by
  simpa using (continuous_id.add continuous_const).tendsto a
```

This shows how Part II's Tendsto statements arise from continuity.

Workflow Example: Derivative Repair

```
#check hasDerivAt_pow
#check HasDerivAt.add

example (x : Real) :
  HasDerivAt (fun y : Real => y ^ 2 + y) (2 * x + 1) x := by
  have hsq := hasDerivAt_pow 2 x
  have hid := hasDerivAt_id x
  simpa using hsq.add hid
```

If this fails, inspect the exact derivative expression and normalize the target.

AI is useful for drafts. Lean is the verifier. Mathlib search repairs the gap.

Good uses of AI:

- propose possible theorem names;
- suggest proof structure;
- explain errors in mathematical language;
- produce a first draft to be checked and corrected.

Suggested Exercises

- Explain `Tendsto u atTop (nhds L)` in epsilon-N language.
- Use `Metric.tendsto_nhds_nhds` to identify the epsilon-delta form of a function limit.
- Prove a simple continuity statement by composing known continuous functions.
- Prove or inspect the derivative of $x \mapsto x^2$ and $x \mapsto x^3 + x$.
- Compute the interval integral of a constant function.
- Repair a deliberately incomplete proof using `#check` and theorem search.

Live Lean: `codeD4/D4_Exercises.lean`